

Xilinx Quick-Start Tutorial

Using Vivado

1. Start the Xilinx Vivado software:

- Open a terminal by right-clicking on the desktop and selecting “Open Terminal”
- Enter the following commands to load Vivado and run it (our default version should be 2019.1):

```
$ module load vivado
$ vivado&
```

- To run a different version of Vivado execute:

```
$ module avail vivado
```

to get a list of available versions, then run:

```
$ module load vivado/<version>
$ vivado&
```

2. Create a new project: **File** → **Project** → **New...**

3. Click **Next**. On the **Project Name** page, enter an appropriate name and location for the project. **Ensure there are no spaces in the project location path; some of the Xilinx tools cannot handle paths with spaces.** Click **Next**.

4. On the **Project Type** page, leave the project type as an RTL project with “Do not specify sources at this time” unchecked and click **Next** to proceed to the **Add Sources** page.

5. Select **VHDL** as the target language and click **Next** twice, skipping the **Add Constraints** page.

6. On the **Default Part** page, configure the project properties as shown in Figure 2 by clicking **Boards** and scroll down to select the **Nexys4 DDR** board. The Xilinx Vivado software will use this board’s layout definitions to later help implement designs. Click **Next**.

- If the board interfaces are not necessary for the project, the part number itself can be selected by clicking **Parts** and selecting the part **xc7a100tcsg324-1**

7. The **New Project Summary** page simply summarizes the project settings. Click **Finish** to exit the wizard and create the project.

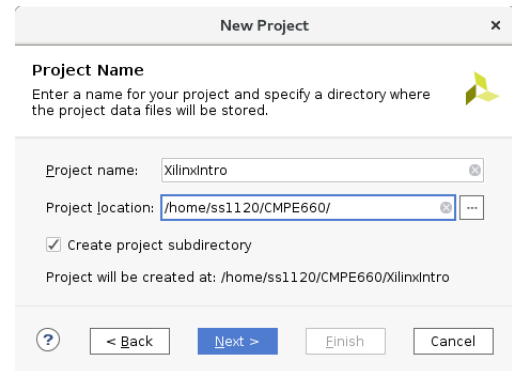


Figure 1: Project Name page of the New Project Wizard

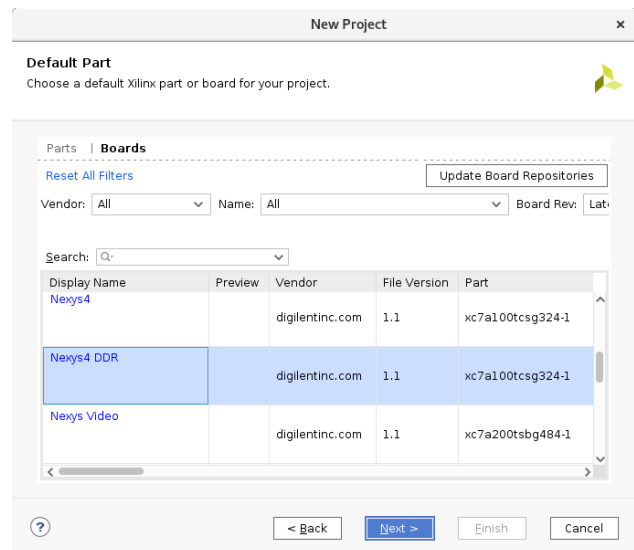


Figure 2: Default Part page of the New Project Wizard

The Xilinx Vivado software provides several panels for managing projects. The screenshot in Figure 4 displays the Flow Navigator panel. The Flow Navigator helps the user manage the design flow of the project.

To the right of the Flow Navigator is the design flow window. This window changes based on step that the user is working on in the design flow. By default, the **Project Manager** design window should be opened when a new project is created. This window displays information such as project summary data, project properties, and the project design sources. The **Project Manager** is how a user will configure the design of the project such as managing source files and project settings.

The project design **Sources** panel lists in a tree format the design files that are referenced by the project. The project itself is the root of the tree under the ‘Design Sources’ directory. Xilinx will detect references between VHDL modules and attempt to organize them logically in this panel.

The **Sources** panel, shown in Figure 3, has three directories:

- The ‘Design Sources’ directory shows all project files that are necessary for synthesis and board programming.
- The ‘Constraints’ directory shows all constraint profiles and their respective constraint files for directing the tools how the design should be synthesized and implemented.
- The ‘Simulation Sources’ directory shows all VHDL files that are relevant to simulation, including Testbenches.

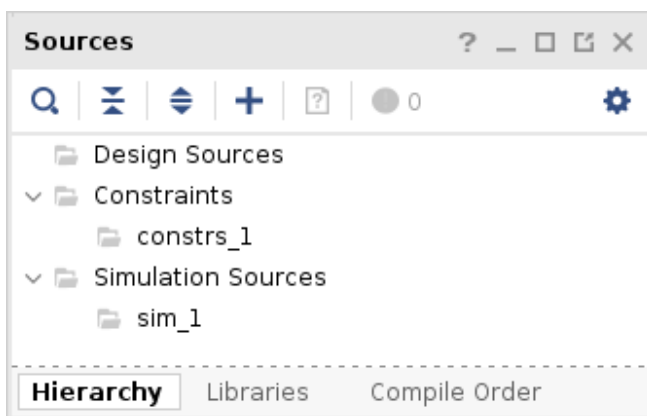


Figure 3: The Sources pane

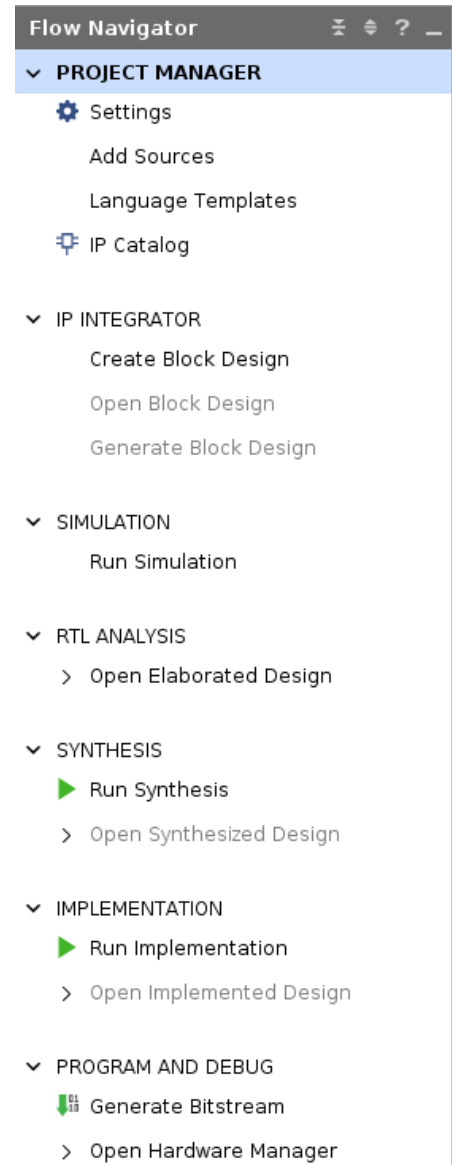


Figure 4: The Flow Navigator pane

8. Add a new VHDL module to the project. Do this using the Add Sources wizard:

Under the **Project Manager** section in the Flow Navigator, click on **Add Sources**. Select **Add or create design sources** and click **Next**.

(Note: The same thing may be accomplished by selecting **File** → **Add Sources...** from the menu bar at the top.)

- On the first page of the Add Sources wizard, select **Create File**, and enter an appropriate file name such as *XilinxIntro* under the file name section. Make sure to select *VHDL* as the file type.

Click **Ok** and then **Finish**.

- The next window that appears is for defining the module ports. The ports will be manually defined in the VHDL code, so leave this blank and click **Ok** and select **Yes** to skip this step exit the wizard.

Xilinx will create the file and add it to the project with some VHDL framework already in place. The file will appear under ‘Design Sources’ in the **Sources** panel.

- Double-check that the project is set up correctly thus far. The Hierarchy in the **Sources** view should appear as shown in Figure 6.

Writing the VHDL Module

- Open or switch to the *XilinxIntro.vhd* file. Add the following library references first:

```
breaklines
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
```

Typically these libraries will be needed for all VHDL code written from here on, so it’s a good idea to include them as a first step.

- Update the entity definition for *XilinxIntro* as follows:

```
breaklines
entity XilinxIntro is
port (
    i_a      : in std_logic;
    i_b      : in std_logic;
    i_s0     : in std_logic;
    i_s1     : in std_logic;
    o_result : out std_logic
);
end XilinxIntro;
```

This tutorial will represent a simple logic unit which performs different logic functions on the inputs, ‘a’ and ‘b’, based on the select inputs, ‘s0’ and ‘s1’.

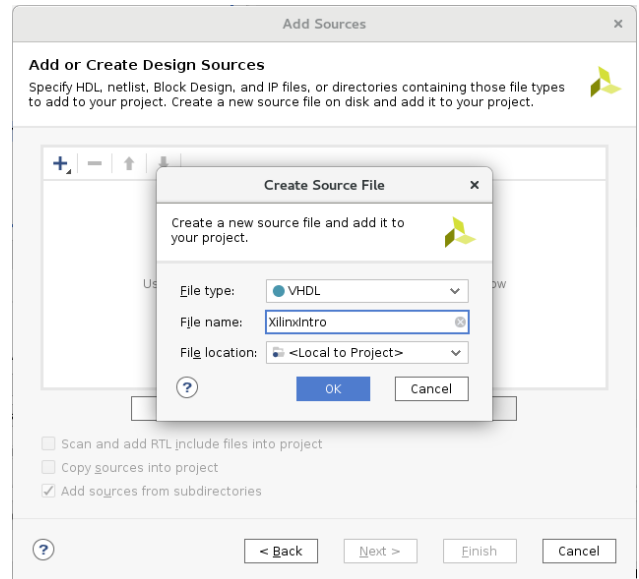


Figure 5: Create Source File window in the Add Sources Wizard

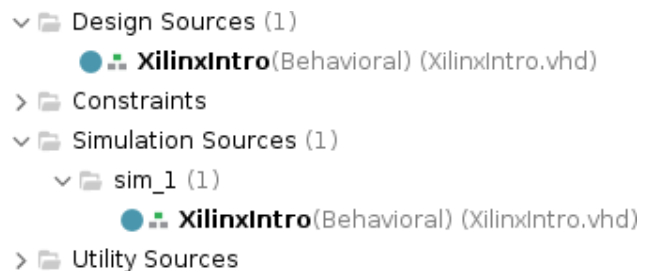


Figure 6: Sources hierarchy so far

14. Add the following internal signal to the architecture for XilinxIntro:

```
breaklines
signal sel : std_logic_vector(1 downto 0);
```

This internal signal will be used to concatenate the select inputs to be used in the case statement.

Remember: signals are defined after the architecture definition, but before the “begin” keyword, i.e. the architecture declarative part.

15. Add the following process to the architecture *after* the begin keyword:

```
breaklines
process (i_a, i_b)
begin
    sel <= std_logic_vector'(i_s1, i_s0);
    case sel is
        when "00" => o_result <= not i_a;
        when "01" => o_result <= i_a and i_b;
        when "10" => o_result <= i_a or i_b;
        when others => o_result <= i_a xor i_b;
    end case;
end process;
```

This process determines which function to perform based on the select inputs. This completes the definition of the XilinxIntro VHDL module. Make sure to save the file before continuing.

Elaborating the Design

16. Elaborate the design by clicking **Open Elaborated Design** under the RTL Analysis section of the Flow Navigator. Click **OK** in the window that appears.

Elaboration will parse the VHDL files and generate a hierarchy of blocks and connections in the design. This hierarchy can be viewed in the generated schematic. If the schematic does not open automatically, click **Schematic** under **Open Elaborated Design**.

17. Look at the schematic. It should be composed of an inverter, AND gate, OR gate, and XOR gate whose inputs are `i_a` and `i_b` and whose outputs are data inputs to a multiplexer. The select input to the multiplexer is the concatenation of the `i_s0` and `i_s1` inputs.
18. Close out of the elaborated schematic view by clicking the X on the Elaborated Design view.

Writing a VHDL Testbench

19. After defining the entity, a **VHDL Testbench** can be added to the project. It is a good idea to add testbenches after defining the entity, as Xilinx Vivado can auto-generate some of the testbench framework that way.

Follow the same procedure as in step 9 to create and add the testbench by selecting the **Add or create simulation sources** option with the Add Sources wizard. As before, add a testbench file after naming it accordingly using the wizard.

20. Verify that the new testbench and previous design source appear under the 'Simulation Sources' directory in the **Sources** view. It should appear as shown in Figure 8.
21. In the XilinxIntro_tb.vhd file, notice that Xilinx has automatically created an entity and architecture for the testbench. Now, a component definition needs to be created for the XilinxIntro module with all the correct ports along with an instantiation of the entity. First start by adding to the top of the testbench architecture the following component declaration along with the following signals that will be used to tie to the module in order to drive the instance and observe the output. To make sure the component ports are written exactly correctly, copy the signals from the entity in the design source file.

```
breaklines
component XilinxIntro is
port (
    i_a      : in std_logic;
    i_b      : in std_logic;
    i_s0     : in std_logic;
    i_s1     : in std_logic;
    o_result : out std_logic
);
end component;

signal a      : std_logic := '0';
signal b      : std_logic := '0';
signal s0     : std_logic := '0';
signal s1     : std_logic := '0';
```

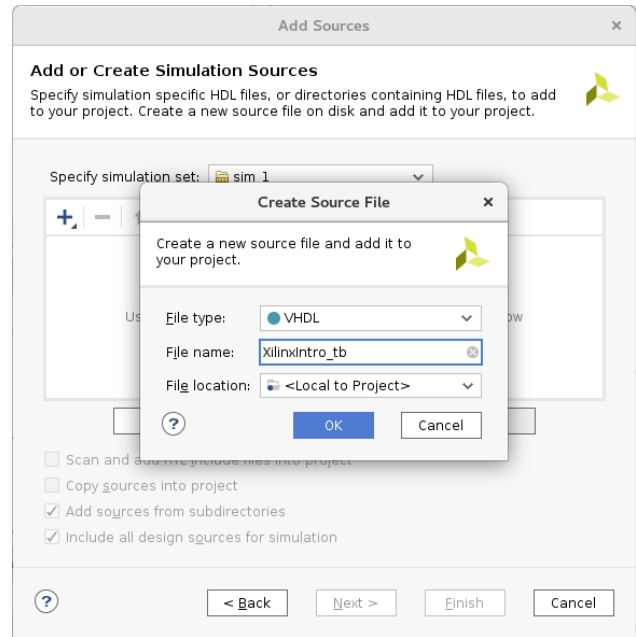


Figure 7: Create Simulation Source File window in the Add Sources Wizard

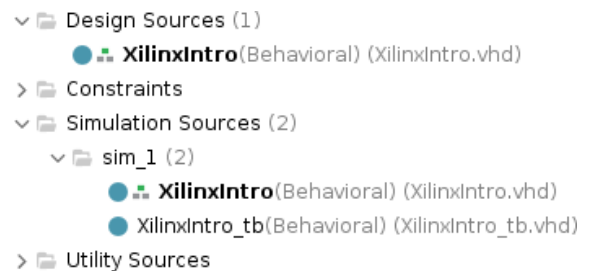


Figure 8: Sources hierarchy so far

```
signal result : std_logic;
```

22. Next, an instance of the XilinxIntro module needs to be instantiated and its ports connected to the new signals. Add the following instantiation after the begin keyword in the testbench architecture.

```
breaklines
    uut: XilinxIntro port map (
        i_a => a,
        i_b => b,
        i_s0 => s0,
        i_s1 => s1,
        o_result => result
    );
```

23. Now a process needs to be added to drive the port signals of the unit under test. Add the process that appears as follows:

```
breaklines
    stim_proc: process
    begin
        s1 <= '0'; s0 <= '0'; -- NOT
        a <= '1';              -- res should be 0
        wait for 10ns;
        a <= '0';              -- res should be 1
        wait for 10ns;
        s1 <= '0'; s0 <= '1'; -- AND
        a <= '0';
        b <= '1';              -- res should be 0
        wait for 10ns;
        a <= '1';
        b <= '1';              -- res should be 1
        wait for 10ns;
        s1 <= '1'; s0 <= '0'; -- OR
        a <= '0';
        b <= '0';              -- res should be 0
        wait for 10ns;
        a <= '0';
        b <= '1';              -- res should be 1
        wait for 10ns;
        s1 <= '1'; s0 <= '1'; -- XOR
        a <= '1';
        b <= '1';              -- res should be 0
        wait for 10ns;
        a <= '1';
        b <= '0';              -- res should be 1
        wait for 10ns;

        wait;
    end process;
```

This stimulus tests various inputs for the circuit. The expected output is shown in the comments. This completes the definition of the testbench. Make sure to save the file before continuing.

Running a Simulation

24. For simulation, the integrated Vivado Simulator will be used. To run the simulation, select **Run Simulation** in the Flow Navigator under the **Simulation** section and then click **Run Behavioral Simulation**. Xilinx will launch Vivado Simulator automatically.

Observe that the outputs change at the exact same time as the input transitions, because there are no delays in a behavioral simulation. Do not be concerned with the actual values yet.

25. Open a new terminal (avoid using the one Vivado was from) and navigate to the project folder. Locate the following directory using `cd`:

```
<projectdirectory>/<projectname>.sim/sim_1/behav/xsim/
```

Take notice of the following files:

- XilinxIntro_tb_vhdl.prj
- XilinxIntro_tb.tcl

These files are scripts created by Xilinx to run simulations using default settings. XilinxIntro_tb_vhdl.prj and XilinxIntro_tb.tcl are auto-generated each time Xilinx creates and then executes a simulation profile. Generally, these do not need to be modified. XilinxIntro_tb_vhdl.prj contains a list of files for simulation compilation. XilinxIntro_tb.tcl contains a list of commands for executing the simulation.

26. Additional simulation configurations are usually desired such as adding more waveforms to the simulation. This can be done by creating a new script with additional commands and instructing the simulator to execute the custom script after the simulation files have been compiled. Create a new `simulation_waveform.tcl` script file in the project directory. Add the following lines to the script:

```
breaklines
#restart simulation
restart

#remove all current waves
remove_wave [get_waves *]

#add desired waves
add_wave /
add_wave /uut/sel

#execute simulation
run 150 ns
```

27. This custom `.tcl` file is used to run alongside simulations to help maintain the desired waveforms and any other additional options for the simulation. The compilation of source files does not need to be maintained as with other simulators as this is maintained by the Vivado simulation design flow. To add the file to the project for post simulation execution, add the file as normal using the **Add Sources** option in the Flow Navigator and choosing the file from the **Add or create simulation sources** menu. Click **Add Files** and change the **Files of type** option at the bottom of the file navigator to **All Files** to be able to select the newly created script file.

28. Close out of the simulation view, and then reopen the behavioral simulation. It should run without errors, but now launches with the `sel` signal traced by the simulator. Now that the TCL file is in the project, changes can be applied by re-executing the simulation without reopening it; this is done using the **Relaunch Simulation** button  at the top of the Project Window. To speed up the initialization of future simulations, the initial runtime of the Vivado simulator can be changed to 0ns, using TCL to manually define simulation length instead. This can be done by opening the **Simulation Settings** and changing the `xsim.simulate.runtime*` option to **0ns** under the **Simulation** tab.
29. Take a close look at the simulation. Is the output correct? From 20 ns to 30 ns, the select signals should select the AND operation, and the data inputs are '0' and '1', why is the output '1'? 60 ns to 70 ns should also be incorrect. The debugger may help to determine why this is.

Debugging

30. Open the `XilinxIntro.vhd` file. Add a breakpoint on the first line of the case statement by clicking the red circle on that line as shown in Figure 9.

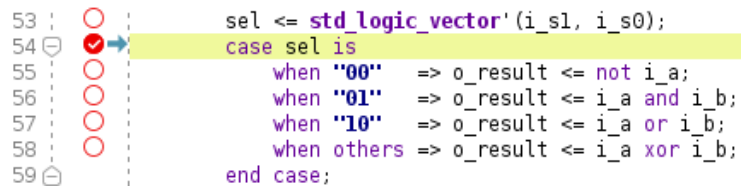
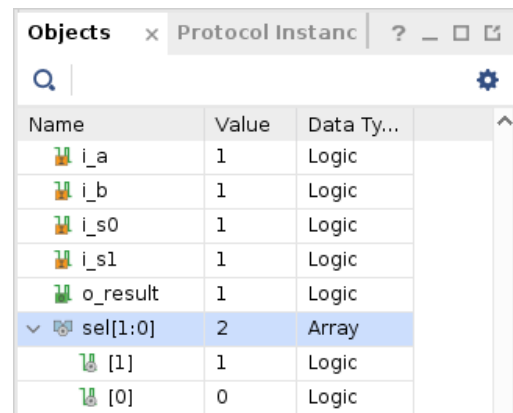


Figure 9: Adding a breakpoint

31. Restart the simulation by clicking the **Restart** button .
32. Run the simulation again by clicking the **Run All** button . The simulation should stop at the breakpoint.
33. At this point, pressing the **Run All** button again will continue the simulation until it reaches the breakpoint again, or the **Step** button  can be used to step through the simulation. Execution will jump between the testbench and the design in the order that each statement is executed by the simulator.

34. Observe the **Objects** pane. The values of the signals and variables are shown at the current time step. Pay attention to how the signals change after each time the breakpoint is hit. Figure 10 shows the state of the signals at the breakpoint set in Figure 9 and shows an instance of the problem.

Hint: The breakpoint is set after the 'sel' signal concatenation statement, just before the case statement is evaluated. Do the values of the select inputs and the 'sel' signal match? Should they? If the 'sel' signal needs to be updated **before** the end of the process, it may be better suited as a **variable**.



Name	Value	Data Ty...
i_a	1	Logic
i_b	1	Logic
i_s0	1	Logic
i_s1	1	Logic
o_result	1	Logic
sel[1:0]	2	Array
[1]	1	Logic
[0]	0	Logic

Figure 10: Objects pane showing values of signals at a breakpoint in the middle of a simulation

35. Make the necessary change to fix the simulation and then rerun the whole simulation. Break-points can be removed by clicking on them again. Note that you may need to edit the TCL file, as variables cannot be added to the waveform view.
36. Close out of the simulation view by clicking the 'X' on the Behavioral Simulation view.

Synthesizing the Design

37. Follow the same procedure as in step 9 to create and add an **Implementation Constraints File** by opening the Add Sources wizard again and selecting the **Add or create constraints**. This file specifies any constraints that are placed on the hardware - including which ports of the VHDL module map to which physical pins on the FPGA. Click **Create File** and provide an appropriate file name like *Nexys4* then click **Ok**. Click **Finish** to create this file. Xilinx will create the XDC file and add it to the project. The file will appear under 'Constraints' in the **Sources** panel.

38. Double-check that the project is set up correctly thus far. The Hierarchy in the **Sources** view should appear as shown in Figure 11.



Figure 11: Sources hierarchy so far

39. Open the User Constraints File *Nexys4.xdc*. If it is not already open, select it from the Hierarchy under constraints.

Add the following lines to the XDC file:

```

breaklines
# ==== LEDs ====
set_property PACKAGE_PIN H17 [get_ports o_result]
set_property IOSTANDARD LVCMOS33 [get_ports o_result]

# ==== Switches ====
set_property PACKAGE_PIN R15 [get_ports i_s1]
set_property IOSTANDARD LVCMOS33 [get_ports i_s1]
set_property PACKAGE_PIN M13 [get_ports i_s0]
set_property IOSTANDARD LVCMOS33 [get_ports i_s0]
set_property PACKAGE_PIN L16 [get_ports i_a]
set_property IOSTANDARD LVCMOS33 [get_ports i_a]
set_property PACKAGE_PIN J15 [get_ports i_b]
set_property IOSTANDARD LVCMOS33 [get_ports i_b]
  
```

These lines define all of the ports on the XilinxIntro entity, how they map to pins on the FPGA, and their attributes. Note that although VHDL is case-insensitive, the case of ports in the constraints file must match the VHDL port declaration.

More information on component pin definitions can be found in the Board Manual (nexys4ddr_rm.pdf) on page 17 under section **Basic IO**. Complete declarations for the XDC file can be found in the sample file: Nexys4DDR_Master.xdc

40. Look at the switches and LEDs on the Nexys4 DDR board. There are small letters and numbers next to them indicating the pins they connect to. Match up the pin numbers with those written in the XDC file. For example, pin J15 is the i_b port and it is the rightmost switch.

41. Often, the XDC file will need to be changed. To make it so the project does not need to be resynthesized when this happens, in the **Source File Properties** pane with the XDC file selected, scroll all the way down to the **Used In** section and uncheck **Synthesis** as shown in Figure 12.

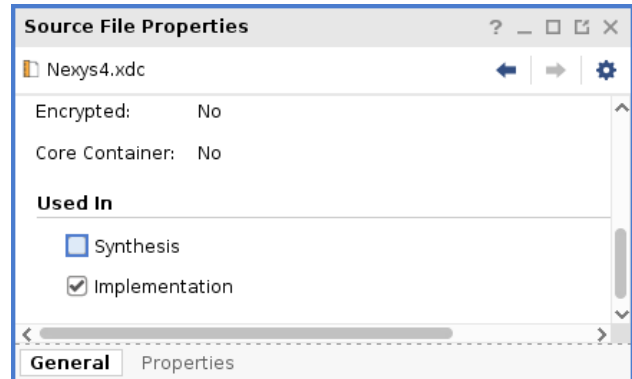


Figure 12: Uncheck Used In Synthesis

42. Synthesize the design. Click **Run Synthesis** under the Synthesis section in the Flow Navigator and click Ok in the Launch Runs window that appears.

Synthesis will compile the VHDL code using Xilinx tools and create a hardware model that is used by later steps.

43. Either click **Open Synthesized Design** and then click **OK** in the window that appears after synthesis OR click **Open Synthesized Design** under the Synthesis section in the Flow Navigator. Open the schematic by clicking **Schematic** under **Open Synthesized Design**.
44. Look at the schematic. There should be a buffer for each of the input and output ports and just one LUT for the entire circuit. Click on the LUT and go to the **Truth Table** tab (the rightmost tab) of the **Cell Properties** pane. Look at the truth table and equation, which should implement the entire circuit.
45. Close out of the synthesis schematic view by clicking the 'X' on the Synthesized Design view.

Post-Synthesis Simulation

46. Xilinx tools support post-synthesis simulation models. It is generally a good idea to run the testbench with the post-synthesis model to ensure correct functionality. To run this simulation, click on **Run Simulation** and execute the **Post-Synthesis Functional Simulation**.

47. This simulation should look correct as well. To do some more careful testing, add the following lines to the end of the testbench, before the last “wait;”.

```
breaklines
    a <= '1';
    b <= '0';
    s1 <= '0'; s0 <= '0'; -- NOT -- res should be 0
    wait for 10ns;
    s1 <= '0'; s0 <= '1'; -- AND -- res should be 0
    wait for 10ns;
    s1 <= '1'; s0 <= '0'; -- OR -- res should be 1
    wait for 10ns;
    s1 <= '1'; s0 <= '1'; -- XOR -- res should be 1
    wait for 10ns;
```

48. Rerun both the post-synthesis **and** behavioral simulations. The post-synthesis should generate the correct results, but the behavioral simulation should be incorrect for the new test cases.
49. Use the debugger on the behavioral simulation again to try and see the problem. Put a breakpoint a few lines before the first new line and step through. If ‘sel’ is now a variable, it should show up as such in the **Objects** pane when the debugger is inside the relevant process. This is useful because variables cannot be added to waveforms.
50. If the simulator steps through each of the statements in the testbench, but stops going back into the design, it means that the process is not getting triggered to start. What triggers a process to start? Is something missing?
51. Fix this problem and rerun both simulations. They should now match and both be correct. It is always a good idea to make sure both behavioral and post-synthesis simulations match before moving on to implementation.

FPGA Implementation

52. Now execute the Implementation Process. Click **Run Implementation** under the Implementation section in the Flow Navigator. Click **Ok** in the Launch Runs window that appears.
53. Either click **Open Implemented Design** and then click **OK** in the window that appears after implementation OR click **Open Implemented Design** under the Implementation section in the Flow Navigator. The device view should open automatically.
54. Ensure the “Routing Resources”  button option is selected to be able to see the routing between all of the resources. Zoom in on a portion of the design to be able to observe the routing through a slice.

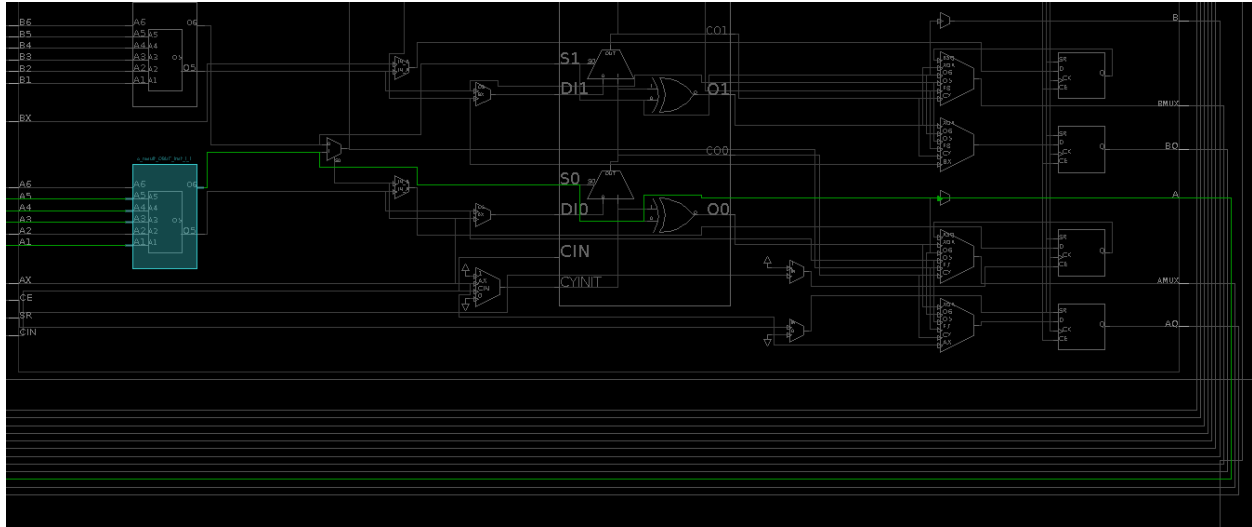


Figure 13: Device view with routes button selected

55. Close out of the device view by clicking the ‘X’ on the Implemented Design view.
56. Xilinx tools also support implementation simulation models. It is also a good idea to run the testbench with this model as well. To run the design using one of these models, click on **Run Simulation** and execute at least once the **Post-Implementation Functional Simulation**.
57. This simulation should already be correct. Execute the **Post-Implementation Timing Simulation** and observe the timing delays that are now present between the input transitions and the output transitions.

The functional models are generated HDL that take into account functional models but ignore timing conditions. The timing models can be run as well but they may take longer to simulate. Note that these options are only available after executing their respective design flow step. In addition, due to the nature of the generated files, Xilinx may change the names of the netlist signals so the current simulation script may not add the intended waveforms. This can be remedied by simply adding the desired waveforms during simulation or the script file can be modified. After executing each simulation model once, the generated simulation models can be found at:

```
<projectname>.sim/sim_1/synth/func/xsim/Testbench_func_synth.vhd
<projectname>.sim/sim_1/impl/func/xsim/Testbench_func_impl.vhd
```

58. Generate a bitstream file for programming the board. Click **Generate Bitstream** under the Program and Debug section in the Flow Navigator click **Ok** in the Launch Runs window that appears.
59. Click **Open Hardware Manager** and then **OK** in the window that appears after generating the bitstream.
60. Connect the FPGA to the computer via the USB programming interface. There is only one micro-USB port on the board which is used for powering the board and JTAG programming as well as for serial communication with the PC. Turn on the power to the board using SW16.

61. Click **Open Target** under Hardware Manager and select **Auto Connect**. This option will launch a hardware server for managing the programming of the board in the background of Vivado. It will automatically detect and connect to the board if the board is properly connected. The **Hardware Manager** window that appears is used to program and debug the designs that are loaded onto the board. The board FPGA will appear under the **Hardware** section if the software properly detected the board.
62. To program the FPGA with the bitstream that was generated, right-click on the FPGA part in the **Hardware** section and select **Program Device** in the menu that appears. In the window that appears, navigate to the bitstream file if it has not been done so automatically. Click **Program** to program the device.

Note: The generated bitstream file should be located in the following folder:

`<ProjectName>.runs/impl_1/`

63. Observe the FPGA board. The rightmost LED (H17) represents the result output and the rightmost four switches control the inputs. Refer back to the XDC file for which switch controls which input and test several combinations to ensure everything worked.

Tutorial 1: Xilinx Quick-Start Tutorial

Student's Name: _____

Demo	Point Value	Points Earned	Date
Part 1: Testbench and simulation (steps 1-29)	25		
Part 2: Debugging - fixing first problem (steps 30-36)	25		
Part 3: Synthesis and simulation - fixing second problem (steps 37-51)	25		
Part 4: FPGA implementation - working demo (steps 52-63)	25		

Demo details:

- Part 1 :
 - Behavioral simulation waveform after modifying the tcl file
 - Elaborated Schematic
 - Answer to the question stated in Step 29: Is the waveform result correct? If not, then identify when (in ns) the waveform is incorrect.
- Part 2:
 - Behavioral simulation from step 35
 - Answer to the following: What was the underlying issue behind this part? What change(s) did you make? And why did your change(s) fix the problem?
- Part 3
 - Inferred schematic from synthesis
 - Post-synthesis functional simulation (before adding extra test cases)
 - Post-synthesis functional simulation (after adding extra test cases)
 - Behavioral simulation (after adding extra test cases)
 - Answer to the following: Why were the behavioral simulation and post-synthesis functional simulation different? How did you fix this issue? Why did that solve the problem?
 - Corrected behavioral simulation (after adding extra test cases)
- Part 4
 - Post-implementation functional and timing simulations
 - Hardware Demo

Submit this grading sheet on myCourses under Assignemnts → Tut1.